
implementation.

Live service: <https://jj-kitchenette-dem9.onrender.com>

Blueprint/service name: jj-kitchenette-dem9

Document Scope

- Functional documentation for customer, admin, and staff workflows.
- Technical documentation for source structure, database tables, configuration, deployment, email, and maintenance.
- Operational guidance for troubleshooting slow loads, broken images, and email delivery.

1. Executive Summary

J&J's Kitchenette is a PHP and MySQL-based online ordering system for a food business. The application supports customer browsing, account registration, cart checkout, delivery or store pickup ordering, admin product management, order processing, reporting, staff permissions, password resets, and email notifications.

The production deployment uses a Dockerized PHP 8.3 Apache runtime on Render, a remote MySQL database on Aiven, and Brevo API email delivery. The current Render Blueprint is named `jj-kitchenette-dem9` and is configured for the Singapore region.

2. Project Objectives

- Provide a customer-facing storefront where users can view menu items, product details, pricing, and recommendations.
- Allow registered customers to manage profiles, saved addresses, carts, orders, and checkout details.
- Provide admins and authorized staff with inventory, product, order, category, customer, report, and settings management.
- Support order confirmation and status update emails using an API-based mail provider suitable for Render hosting.
- Deploy reliably to a cloud environment with documented environment variables and operational troubleshooting steps.

3. System Users and Roles

Guest	Browse homepage, menu, product pages, contact page, and location page.
Customer	Create account, log in, manage profile and addresses, add cart items, checkout, and view orders.
Staff	Access admin features granted through staff permissions such as products, inventory, categories, or orders.
Admin	Full access to dashboard, products, orders, customers, reports, categories, settings, staff users, and mail testing.

4. Application Architecture

The system is organized as a traditional server-rendered PHP application. Pages include PHP controller logic at the top, database queries through `mysqli`, and HTML views rendered directly in the same file. Shared utilities are placed in `includes`, while admin pages are grouped under the `store` directory.

Public storefront	index.php, menu.php, product.php, contact.php, location.php
Customer account	account.php, login.php, account/profile.php, account/orders.php
Cart and checkout	cart.php, checkout.php, process_order.php, order_success.php
Admin/store	store/dashboard.php, products.php, orders.php, order_detail.php, reports.php, settings.php
Shared services	db.php, includes/session.php, includes/images.php, includes/mailer.php, includes/password_reset.php
Deployment	Dockerfile, render.yaml, .htaccess, composer.json

5. Customer-Facing Features

5.1 Homepage and Menu

- Homepage shows a dynamic hero, menu categories, featured items, product count, and category count.
- Menu page supports category filtering, search, availability display, product cards, and AJAX-style result updates.
- Product images use `appImageUrl` so local files load directly and missing files can fall back through `image.php` or `default.png`.

5.2 Product Details and Cart

- Product page presents variants, options, inventory-aware quantity selection, and add-to-cart/buy-now actions.
- Cart page lists selected items, prices, quantities, delivery fee estimate, free-delivery progress, and recommendations.
- If no saved address exists, the cart shows an address reminder before checkout.

5.3 Checkout

- Checkout supports cart checkout and direct checkout from a product page.
- Payment methods are Cash on Delivery and Store Pick Up.
- Cash on Delivery is restricted to Tangub City in the user-facing warning text.
- Address creation is supported through an AJAX modal during checkout.
- Final order submission validates stock, creates order and `order_items` records, deducts inventory, clears cart, and sends email.

6. Admin and Staff Features

Dashboard	Shows summary metrics, recent orders, and low-stock/active product information.
-----------	---

Products	Create, edit, delete, and preview products, images, variants, options, inventory, and status.
Orders	Review orders, update status, cancel unavailable items, and trigger order update emails.
Customers	Admin-only customer listing and detail management.
Categories	Manage menu category records.
Reports	Admin-only operational and sales reporting.
Settings	Store settings, staff users, password management, and mail test diagnostics.

Staff access is controlled by `staff_permissions`. Admin users bypass staff permission checks, while staff users only see and access modules granted to them.

7. Database Design

The database is MySQL-compatible and is currently hosted on Aiven. The repository includes `jj_kitchenette_dump.sql`, which defines the core schema.

<code>users</code>	Stores customer, admin, and staff account records.
<code>roles / permissions / role_permissions</code>	Role and permission structure for account access.
<code>staff_permissions</code>	Fine-grained staff access to inventory, products, categories, and orders.
<code>addresses</code>	Saved customer delivery addresses.
<code>products</code>	Product identity, handle, title, body, category, status, and timestamps.
<code>product_variants</code>	SKU, options, price, inventory, and variant-level product details.
<code>product_images</code>	Image paths and optional image blob/mime storage.
<code>product_options / product_option_values</code>	Configurable option names and values for product variants.
<code>cart</code>	Customer cart line items.
<code>orders</code>	Order header, customer snapshot, fulfillment, status, subtotal, delivery fee, and total.
<code>order_items</code>	Order line items with product snapshot, options, SKU, price, quantity, status, and image path.

order_notes	Operational notes attached to orders.
password_reset_codes	Temporary password reset and staff invitation codes.
auth_tokens	Session/token support table from the schema dump.

8. Configuration and Environment Variables

Runtime configuration is supplied through Render environment variables. Sensitive values are marked `sync: false` in `render.yaml` and must be entered in the Render dashboard.

APP_URL	Public application URL: <code>https://jj-kitchenette-dem9.onrender.com</code>
APP_TIMEZONE	Application timezone, currently Asia/Manila.
APP_SESSION_LIFETIME	Session cookie and garbage collection lifetime.
DB_HOST, DB_PORT, DB_DATABASE	Aiven MySQL connection endpoint and database name.
DB_USERNAME, DB_PASSWORD	Aiven database credentials.
DB_SSL_CA/DB_SSL_CA_CONTENT	SSL certificate path or inline CA content for Aiven TLS connections.
DB_PERSISTENT	Optional toggle, persistent connections are enabled unless set to false.
MAIL_FROM_EMAIL, MAIL_FROM_NAME	Verified sender details used by Brevo or SMTP fallback.
BREVO_API_KEY	Brevo API key for HTTPS-based email delivery.

9. Deployment

The application is deployed as a Docker web service on Render. The Dockerfile installs PHP 8.3 Apache, mysqli, PDO MySQL, curl, Apache rewrite/expires/headers modules, and Composer dependencies.

Render service: `jj-kitchenette-dem9`
Region: `singapore`
Runtime: `Docker`
Port: Render `PORT` env var, default fallback `10000`
Main URL: `https://jj-kitchenette-dem9.onrender.com`
Database: Aiven MySQL in Singapore
Email: Brevo API through PHP curl

Deployment Steps

1. Push code changes to GitHub main branch.
2. Render detects the change and builds the Docker image.
3. Composer installs PHPMailer dependencies in the vendor stage.
4. Apache starts with the Render PORT value.
5. The app reads Render environment variables, connects to Aiven, and serves PHP pages.
6. After deployment, test homepage, menu, checkout, admin login, and /store/mail-test.php.

10. Email Delivery

Render blocks outbound SMTP ports commonly used by Gmail SMTP, so the project uses Brevo's HTTPS API for production email delivery. The mailer chooses Brevo when BREVO_API_KEY is present; otherwise it falls back to SMTP.

Customer password reset	Customer forgot-password flow.
Staff invitation/reset	Admin or Staff password setup/reset flow.
Order confirmation	Successful order creation in process_order.php.
Order update	Admin order status updates and cancellation workflows.
Contact message	Contact page form submission to the store email.
Mail test	Admin-only /store/mail-test.php diagnostic page.

- Set BREVO_API_KEY in Render.
- Set MAIL_FROM_EMAIL to a Brevo-verified sender.
- Use /store/mail-test.php after each deployment or credential change.
- If email fails, inspect Render logs for lines beginning with Mail error.

11. Performance and Caching

-
- Homepage anonymous responses use a short server-side HTML cache and X-Home-Cache HIT/MISS headers.
 - Static assets use .htaccess Cache-Control headers and Apache expires/headers modules.
 - Images are served directly from local uploads when available, with image.php fallback for missing local files or blob-backed images.
 - Database connections use mysqli persistent connections for remote non-local hosts unless DB_PERSISTENT=false.
 - Render free services can still have cold starts after inactivity; upgrading to an always-on plan removes that class of delay.

12. Security Considerations

- Sessions are initialized through `includes/session.php` with `secure`, `HttpOnly`, `SameSite=Lax` cookie parameters.
- Passwords are hashed with `password_hash` and verified with `password_verify`.
- Admin and staff pages use `store/includes/admin-auth.php` guards.
- Staff capabilities are limited by `staff_permissions`.
- Sensitive credentials must remain in Render environment variables, not committed to source control.
- Password reset codes are hashed and expire after 15 minutes.

13. Testing and QA Checklist

Public pages	Homepage, menu, product details, contact, location load correctly.
Auth	Customer signup, confirm password, login, logout, forgot password.
Cart	Add item, update quantity, remove item, recommendation images, no-address note.
Checkout	Cash on Delivery address requirement, Tangob City warning, Store Pick Up, order confirmation.
Admin	Dashboard, product CRUD, order status changes, staff permissions, reports.
Email	Mail test, password reset, order confirmation, order update, contact form.
Deployment	Render build, DB connection, APP_URL, Brevo key, Aiven SSL, static cache headers.

14. Troubleshooting Guide

Slow first load	Render free service cold start. Use paid always-on instance for fastest first request.
Slow warm dynamic pages	Check Render and Aiven regions, DB connection time, query load, and persistent connections.
Broken product image	DB <code>image_path</code> points to a file missing from uploads. Use <code>applImageUrl</code> fallback or re-upload image.
Emails not sending	Confirm <code>BREVO_API_KEY</code> , verified <code>MAIL_FROM_EMAIL</code> , PHP curl, and Render logs.

Blueprint sync failed	Check render.yaml service name, region support, and env var syntax.
Checkout blocked	Cash on Delivery requires a saved Fungub City address, Store Pick Up does not require address.

15. Maintenance Notes

1. Before editing, check git status and avoid overwriting unrelated changes.
2. After PHP changes, run `php -l` on edited PHP files.
3. Push every completed change to GitHub so Render can deploy.
4. Keep Render environment variables synchronized with `render.yaml` placeholders.
5. Use `/store/mail-test.php` after changing mail provider credentials.
6. Keep Aiven and Render in nearby regions, preferably Singapore for this deployment.
7. Back up the MySQL database before schema changes or large product imports.

16. File Reference

<code>index.php</code>	Homepage with featured products, categories, stats, and cache behavior.
<code>menu.php</code>	Searchable and filterable menu listing.
<code>product.php</code>	Product detail page with variants and cart actions.
<code>cart.php</code>	Cart summary, delivery fee estimate, recommendations, and checkout link.
<code>checkout.php</code>	Address, payment, item review, totals, validation, and modal confirmation.
<code>process_order.php</code>	Order creation, inventory deduction, cart clearing, and confirmation email.
<code>store/</code>	Admin and staff management area.
<code>includes/</code>	Shared session, image, mail, password reset, order email, and cancellation helpers.
<code>assets/css/</code>	Public, account, checkout, cart, product, menu, profile, and admin stylesheets.
<code>uploads/</code>	Product and default image assets.
<code>Dockerfile</code>	Production runtime build instructions.
<code>render.yaml</code>	Render Blueprint Configuration.
<code>jj_kitchenette_dump.sql</code>	Database schema dump.

17. Appendix: Suggested Future Improvements

- Add automated integration tests for checkout and admin order status changes.
- Move schema changes into formal migration files instead of runtime CREATE/ALTER checks.
- Add image resizing/compression at upload time to reduce bandwidth.
- Add health check endpoint for Render monitoring.
- Add a formal README plus developer setup guide alongside this PDF.
- Add structured application logs for mail, order, and database events.